

El medio que falta

Hyphae en el residual · de la crítica al contrato de desarrollo

Spec de trabajo · v2.2 · 29 de agosto de 2026

No es interpretabilidad post-hoc. No es un dashboard. No es un manifiesto contra las GPU.

CUDA y ROCm proponen. `apply()` ejecuta política constituida por Hyphae.

Nada es durable, exportable ni citable sin receipts.

INVARIANTE

*El único kernel con derecho a escribir h ejecuta política constituida por Hyphae.
Ningún `optimizer.step()` ni export de checkpoint ocurre sin los receipts del paso.*

CONTENIDO

El contrato, no la lámina

Este documento es la spec de trabajo cerrada al 29 de agosto de 2026. Consolida v2 y las addenda v2.1 / v2.2. No es un paper. No relitiga pieza por pieza la historia: el registro de correcciones vive al final, porque un sistema no es confiable cuando borra sus errores.

- 00 · Qué es, qué no es, invariante
- 01 · Cinco leyes
- 02 · Catálogo: slots y features
- 03 · pack, allocate, veredictos
- 04 · apply_ref y el cero con signo
- 05 · Ley 0 y el baseline parallel
- 06 · Autoridad, durabilidad, loop
- 07 · Cadena del paso y export
- 08 · Wire format BindingMin
- 09 · T1, T2 y el modelo de amenaza
- 10 · Presupuestos y manifiesto
- 11 · Métricas que sí falsan
- 12 · Construcción y lista negra
- 13 · Registro de correcciones

00

Qué es, qué no es

Qué es. El contrato de desarrollo para un transformer cuyo residual no puede mutar sin emitir un hecho, y cuyos pesos no pueden volverse citables sin receipt. Hyphae no es el modelo: es quien firma.

Qué no es. Interpretabilidad post-hoc, un dashboard, un SAE sobre activaciones, un manifiesto contra las GPU, zkML de cada matmul, ni el foro «internet muerto» mezclado con este runtime.

Estado. Fase de desarrollo. El objeto interno tiene nombre (binding) y matemática que un proceso CPU puede ejecutar en frío. Hyphae verificado: v2.2.0 (release-v2.2.0-crates, 5bd8afb). allocate, proof batch async y prev/next nativos por record siguen siendo capacidades de aplicación.

Invariante

CUDA y ROCm proponen. El único kernel con derecho a escribir h ejecuta política constituida por Hyphae. Ningún optimizer.step() ni export de checkpoint ocurre sin los receipts del paso.

01

Cinco leyes

Ley	Enunciado	Si se viola
0. Funcional	layer propone. No escribe h. Attn y mlp leen el mismo h normalizado y suman en un buffer propio.	El += anónimo vuelve por la puerta del framework.
1. Identidad	El camino de escritura nace en cero (ReZero/Fixup). H(h ₀) se commitea como ancla del microbatch.	Init gaussiano como constructor: no hay sujeto ni ancla.
2. Delta nombrable	Toda escritura a h es un binding (slot, feature, mag, policy_id).	Smearing anónimo en R ⁿ . El medio vuelve a ser sopa.
3. Overflow visible	Colisión = hecho. Silencio (slot no propuesto) ≠ OVERFLOW ≠ ABORT.	Superposición invisible. El invariante es decorativo.

Ley	Enunciado	Si se viola
4. Commit o silencio	Solo apply() escribe h. Durable / export / cita exige receipts.	El modelo avanza o se exporta en silencio. El molde ganó.

02

Catálogo: slots y features

El residual es un bus. El slot es un puerto de escritura: un subespacio contiguo de $d_{\text{model}} = S \times d_{\text{slot}}$ (fase 1: $S = 64$). La feature es una fila de un codebook aprendido $C \in \mathbb{R}^{N_f \times d_{\text{slot}}}$, con $N_f \in \{4096, 16384, 32768\}$ prerregistrado. Home de fase 1: $\sigma(f) = f \bmod S$. Colisión: dos candidatos commiteables en el mismo (pos, slot). Gana $|mag|$; el perdedor es OVERFLOW. Cambiar la política es un hecho, no un hotfix.

Eso es la capacidad de escritura. Por token, por capa, como máximo $\min(k, S)$ writes. Cada hoja de d_{slot} dims queda cuantizada a un código del subdiccionario $\lfloor N_f / S \rfloor$. No es un denso disfrazado.

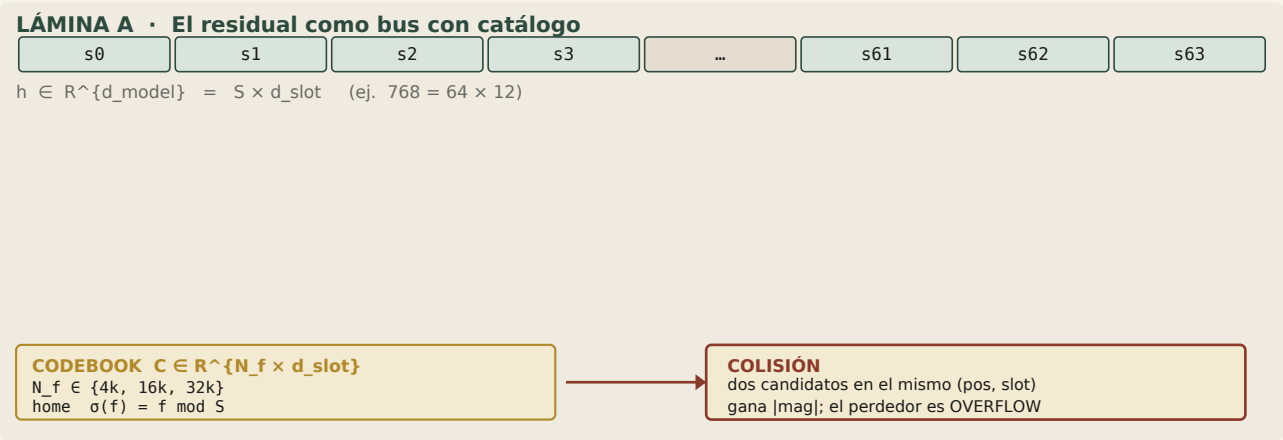


Figura 1. El residual partido en puertos; el codebook nombra lo que cruza cada puerto.

03

pack, allocate, veredictos

pack — de $\delta_{\hat{}}$ a candidatos

layer : $h \mapsto \delta_{\hat{}} \in \text{bf16}^{B \times T \times d_{\text{model}}}$. Nada que no salga de pack entra a allocate. Fase 1, cerrada:

Parámetro	Valor fase 1
Home	$\sigma(f) = f \bmod S$
Score	$\langle \text{leaf}_S, \hat{C}[f] \rangle$ con $\hat{C} = C / (\ C\ _2 + \epsilon)$, $\epsilon = 2^{-14}$
Reducción	secuencial por índice creciente, fp32, sin FMA
mag propuesto	el score; se guarda como bits bf16 aplicados
k	$\{1, 2, 4, 8\}$ — un run, un k
Top-k	global por token sobre los N_f pares legales
Desempate	mayor score → menor feature → menor slot

Con $\text{init} = 0$, en el step 0 todos los scores son 0 y el desempate commitea los mismos features $0..k-1$ para todos los tokens. El aux de balanceo lo ve entero. Es consecuencia del empate total, no un bug. Diversidad de arranque = otro POLICY (`write_path.init = eps_bf16` o tie-break por hash), no un hotfix.

Cuatro destinos

LÁMINA G · Cuatro destinos de un candidato

COMMIT entra al mask STE apply escribe (ceros: no-op + elide)	OVERFLOW perdió (pos, slot) journalizado grad 0 + aux	ABORT nonfinite mag >max journalizado grad 0 + aux	silencio slot no propuesto no hay hecho no hay journal
---	---	---	--

Figura 2. COMMIT, OVERFLOW, ABORT y silencio no son sinónimos.

ABORT de fase 1, solo dos reglas, evaluadas sobre el bf16 del binding: $\text{nonfinite}(\text{mag}) \rightarrow \text{ABORT}$ y $|\text{mag}| > \text{mag_max} \rightarrow \text{ABORT}$. $\text{mag} == \pm 0$ no aborta. Por token, por capa, antes de elidir ceros: $\# \text{COMMIT} + \# \text{OVERFLOW} + \# \text{ABORT} = k$. El manifiesto prohíbe $N_f < k$.

Backward

mag es continuo: STE a través del bf16. El índice feature es discreto: el backward deposita en $C[f]$ vía la normalización. Top-k y colisión no son diferenciables: mask duro sobre el conjunto commiteado, no sobre el top-k. Aux MoE-style sobre la distribución empírica de feature y slot. Además se monitorea $\Sigma |\text{mag}|$ por feature — con apply normalizado, la energía del catálogo es visible en los bindings.

04

apply_ref y el cero con signo

bf16 nativo de CUDA y ROCm promociona, fusiona y redondea distinto. T1 cross-vendor sobre la ALU del driver era el nuevo error de hashes no homomórficos. apply es una referencia C/Rust enlazada igual en el trainer y en el verificador.

```
n^[f] = rnd_fp32( C_fp32[f] / (||C_fp32[f]||₂ + ε) )
leaf' = rnd_bf16( leaf_fp32 + rnd_fp32(mag_bf16) * n^[f] )

si mag_bf16 == ±0:
  leaf' = leaf # copia bit-idéntica; la suma NO se ejecuta
```

Promoción bf16→fp32 por extensión de mantissa. Producto y suma fp32 IEEE, round to nearest even, sin FMA. Downcast bf16 RNE. Denormals-to-zero off. Orden de aplicación: slots crecientes; tokens row-major (b, t). Con la normalización dentro de apply, $\| \text{leaf}' - \text{leaf} \| = |\text{mag}|$ a redondeo: el binding se autodescribe.

El caso cero no es «h+0, da igual». En IEEE, $(-0) + (+0) = +0$. Si el device aplicara la suma y T1 elidiera el binding, un leaf -0 flippearía un bit y T1 mataría un run honesto. Por eso la identidad es definición, no accidente. El forward del kernel copia; el backward es un Function cuyo espejo simbólico $h + \text{mag} \cdot \hat{C}$ conserva STE (como α en ReZero). Los ceros commiteados se eliden del WAL antes del hash T2.

Reproducibilidad prometida: same-binary, CPU contra device, bit a bit. No prometida: génesis de Δ_{hat} entre NVIDIA y AMD, matmuls del bloque, TF32. El binding declara device. Si un backend no puede hospedar la referencia, no entra al harness.

05

Ley 0 y el baseline parallel

Ningún op dentro de layer recibe h mutable. Eso no es un accidente de API: empujó la topología a un parallel block (GPT-J / PaLM). LN lee h; attn y mlp leen el normalizado; ambos suman en un buffer propio; ese buffer es la entrada de pack. No hay $h = h + \text{attn}(\dots)$ intermedio.

El hecho nombra la suma. Attn y mlp que se cancelan en un slot no dejan dos hechos: dejan menos energía en delta_hat. Fase 1 no atribuye subcapa. Quien quiera esa atribución declara dos apply, dos catálogos, un manifiesto hijo.

El baseline denso obligatorio comparte la topología: parallel-block, misma lectura de h, suma densa sin catálogo. baseline.block = parallel. Comparar contra un secuencial clásico mezcla catálogo y topología; se puede reportar aparte y no participa del umbral.

Test de no-mutación: snapshot por bytes (h.clone() pre-layer contra h post-layer), nunca id() de Python. Perfil harness.alias_check ∈ {always, t1_rate, off}. Default de run: t1_rate. CI: always.

06

Autoridad, durabilidad, loop

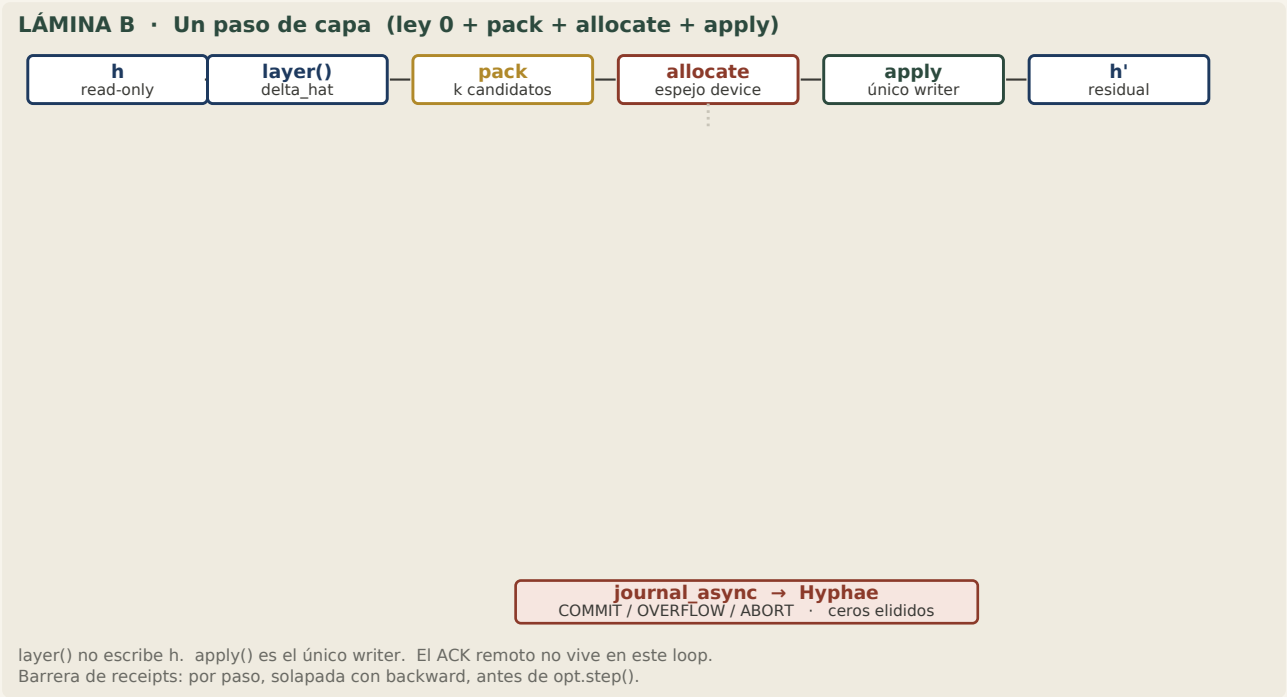


Figura 3. El loop de capa. El device no espera a Hyphae.



Figura 4. Quién decide, quién transporta, quién firma.

En training hay un escritor por instancia Hyphae (un proceso, un data directory, lock de OS). El espejo no arbitra conflictos en caliente: ejecuta política determinista. wait_receipts(step) se solapa

con el backward. Timeout = barrier.budget_ms. Si no hay CommitReceipt: abort del run, dump del ring, no-op sobre opt.step(). No hay degradación a «seguir sin ledger». Reanudación = run hijo que cita el último head.

```
h = embed(batch)
commit_async(H(h), tag='anchor') # cuenta en d2h.max
for layer in model:
    delta_hat = layer(h) # h intacto
    cand = pack(delta_hat, C_step, k)
    verd = allocate_local(cand, policy) # ±0 → COMMIT
    h = apply(h, verd.committed) # ±0 → copia bits
    journal_async(elide_zeros(verd)) # antes de T2
    loss = head(h) + aux_balance(verd_all)
    loss.backward() # mask = committed
    wait_receipts(step) # fail-stop
    opt.step()
    renorm_or_reset_C(policy) # CODEBOOK_RESET si toca
    commit_step_chain(C, policy_id) # H(C) ya limpio
    if step % seal_every_k == 0: commit_seal(θ)
```

07

Cadena del paso y export

LÁMINA E · Cadena del paso



Figura 5. POLICY constituye. STEP hashea C. SEAL hashea θ . EXPORT cita.

La política es un record Hyphae, no un if en el kernel. Bindings posteriores citan policy_id. Un binding sin él es inválido. Fase 1 elige un mecanismo de C — STE+optimizer o EMA con hecho CODEBOOK — no los dos. EMA silencioso en un hook de PyTorch está prohibido.

Después de opt.step() o del update CODEBOOK, y antes de $H_{\text{canónico}}(C)$: se mide $\|C[f]\|_2$ pre-renorm. Si $n < \min_{\text{norm}}$ (default 2^{-8}), el feature está muerto y se resetea. Si no, se renormaliza a 1. $H_{\text{canónico}} = \text{SHA-256}$ de shape || dtype || raw row-major little-endian. Nada de pickle. Nada de «resumen por tensor».

Muerte también por ventana de uso: codebook.dead_window steps (default 100) sin COMMIT de mag $\neq 0$. El reset emite CODEBOOK_RESET. El verificador no lo recomputa (es data-dependent); lo evidencia $c_{\text{prev}} \rightarrow c_{\text{next}}$, misma epistemología que opt.step().

θ no se hashea cada paso: se sella cada seal.every_k_steps y siempre antes de EXPORT. Entre sellos, la mutación está evidenciada (cadena de STEP: lr, grad_norm, policy_id, orden) y por los artefactos de build, no hasheada. T2 no pesca un opt.step() distinto que loguee los mismos escalares. Modelo de amenaza: fallo de implementación bajo el artefacto citado, no adversario. Adam (m, v), RNG y orden del dataloader quedan fuera de fase 1, declarados.

El artefacto citable no es el .pt crudo. Es EXPORT step_id w=H(θ) c=H(C) head receipt sobre tensores en layout documentado. Si alguien reexporta con torch.save y el hash cambia, no está en la cadena.

08

Wire format BindingMin

LÁMINA F · BindingMin 16 bytes little-endian

feature 2B	slot 1B	dev 1B	mag_bf16 2B	layer 2B	pos 4B	cand 2B	verd 1B	rsn 1B
verdict: COMMIT=0 OVERFLOW=1 ABORT=2 reason: none=0 nonfinite=1 mag_overflow=2 policy=3								

Figura 6. Dieciséis bytes. Sin string de id. La identidad es la posición en T2.

mag son bits bf16 crudos — no IEEE binary16. slot es derivable de feature bajo el home de fase 1 y se conserva deliberadamente: invariante $\text{slot} == \text{feature} \bmod 5$ en ingest, y schema estable para routing aprendido. step_id, policy_id, run_id, microbatch_id viven en el STEP / head de capa, una vez, no por hecho. Perfil t1_5 añade $\text{prev}[32] + H(\text{pre_leaf}[32]) = 80 \text{ B}$. Subir N_f o S es cambio de schema = SPEC_AMEND.

09

T1, T2 y el modelo de amenaza

LÁMINA D · T1 replay y T2 cadena

T1 · consistencia (muestra 1/N)

```
derrama h0 (~3 MB auditado)
bindings commiteados, orden total
C_step (c_next del STEP)

h' = apply_ref(h0, B, C)
¿ H(h') == head ?

same-binary · CPU · sin GPU
```

T2 · integridad del WAL (siempre)

```
head_{[]+1} = H(head_[]
                || bindings_[]
                || meta_[])

meta: layer, n_persistidos,
      n_elididos, policy_id

flip de 1 bit → localiza eslabón
```

Figura 7. T1 observa una muestra. T2 sella el WAL entero.

T1 prueba consistencia $\text{apply} \leftrightarrow \text{journal} \leftrightarrow \text{codebook}$ del STEP sobre la fracción 1/N de microbatches, selección $\text{hash}(\text{run_id}, \text{step_id}, \text{mb}) \bmod N == 0$. Un T1 verde no es omnisciencia del residual. El invariante en el complemento es el binario citado en RUN_START, no la muestra.

Fase 1 audita fallos de implementación, no adversarios. La selección T1 es predecible desde el step 0. Un trainer malicioso podría desviarse solo en el complemento. Commit-then-reveal de la semilla = fase posterior. El mismo modelo cubre el hueco entre SEALs y el hecho de que el WAL, con ceros elididos, no reconstruye el mask de STE: no hay resume verificado en esta fase.

T2 corre siempre. Un flip de 1 bit en el WAL persistente rompe la cadena y se localiza al eslabón. meta_[] incluye layer, n_persistidos y n_elididos para que una capa vacía del step 0 (todos los ceros elididos) no sea un eslabón amorfo.

El binario como hecho

RUN_START cita cuatro digestos. Falta uno ⇒ el run no arranca.

Campo	Qué pinnea
build.apply_ref_hash	SHA-256 del .so / objeto de apply_ref (idéntico en trainer y verify)
build.harness_commit	commit git del harness; árbol limpio, no working dirty
build.torch_wheel	nombre + versión + digest del wheel
build.backend_wheel	nombre + versión + digest (cuda / rocm)

Un quinto campo opcional build.image_digest si hay contenedor; no sustituye los cuatro. El hash monolítico «del proceso Python» está deprecado y el harness lo rechaza.

10

Presupuestos y manifiesto

Cotas, no prosa. Con $\beta = 16\text{ B}$ (mínimo) o 80 B (t1_5):

```
N_hechos_fwd = L × N_tokens × k
bytes_bindings ≤ N_hechos_fwd × β # baja con ceros elididos
bytes_step = bindings + STEP + anchor + POLICY? + CODEBOOK? + SEAL?
```

Ejemplo: L=12, N_tokens=2048, k=4 → 98 304 hechos → 1.5 MiB/paso ($\beta=16$) o 7.5 MiB (t1_5). Caso grande L=32, N_tokens=8192, k=8 → 2 097 152 hechos → 32 MiB/paso. Por eso k, L y el perfil entran al manifiesto juntos.

Campo	Fase 1
wal.max_bytes_per_token / per_step	cota; violación mata la rama
wal.elide_zero_commits	true
d2h.max_bytes_per_step	incluye framing y anchor H(h ₀)
d2h.overlap	backward
barrier.budget_ms	fail-stop
seal.every_k_steps	cadencia de H(θ)
t1.sample_rate	1/N prerregistrado
t1.max_h0_bytes_per_day	spill de auditados
write_path.init	0 (eps_bf16 = otro POLICY)
codebook.post_step / min_norm / dead_window	renorm / 2 ⁻⁸ / 100
layer.write_units / sublayer_facts	1 / false
binding.profile	min t1_5
harness.alias_check	t1_rate
baseline.block	parallel
build.apply_ref_hash + 3	obligatorios en RUN_START

11

Métricas que sí falsan

Se mide	Regla
Gap de perplexity vs denso parallel a FLOPs iguales	FLOPs cuentan layer + pack + apply_ref. Umbral X% prerregistrado. Mata la rama si se cruza.
T1 al 100 % de mutaciones inyectadas en la muestra	No se reclama sobre el complemento. Inyección = harness.
T2 al 100 % de flips de 1 bit en el WAL persistido	Integra el ledger. No mide el modelo.
Jaccard seed-a-seed de bindings	Solo tras alinear el codebook (Hungarian por slot hogar). Sin alinear no se reporta.

Se mide	Regla
Entropía de C y $\Sigma \text{mag} $ por feature	Colapso = mata la rama. El colapso por norma ya no es invisible.
Overhead de pared	D2H + barrera + journal + SEAL contra las cotas del manifiesto.
Humano cita un binding y reencuentra su proof	Con $ \text{mag} $ = magnitud real del hecho.

No se mide todavía: MMLU, chatbot arena, «transparencia percibida», que una neurona signifique un concepto, paridad numérica NVIDIA/AMD, zkML, que Hidden Layer 2 se vuelva un mapa semántico.

12

Construcción y lista negra

Antes del primer run que pueda perder, en este orden:

- 1. Ley 0 en el bloque + test de no-mutación por bytes.
- 2. apply_ref con $\hat{C}$, renorm y no-op del cero + kernel device bit-idéntico.
- 3. Schema BindingMin con verdict/reason y mag_bf16.
- 4. pack fase 1 con k, métrica y reducción fijados.
- 5. Record POLICY con abort_rules instanciadas + espejo.
- 6. STEP/CODEBOOK con H(C) por paso + SEAL de θ por cadencia.
- 7. Manifiesto con cotas, sample_rate y los cuatro build.* escritos antes del step 0.
- 8. Verificador T1 same-binary + T2, proceso aparte, CPU.
- 9. El mismo harness en CUDA y ROCm. Si solo cierra en uno, se eligió vendor.

Lista negra

- Loguear cada MAC o activación «por si acaso»; un slot basura para que el loss baje.
- if nvidia / if amd en la costura de apply.
- Esperar proof o ACK remoto por capa en el hot path.
- apply nativo del vendor sin match bit a bit con apply_ref.
- Ejecutar la suma IEEE para $\text{mag}=0$ y llamarla identidad.
- Elidir ceros después de computar el head T2.
- $\text{mag}=0 \rightarrow \text{ABORT}$ (deroga C7c). STE apagado sobre el zero-init.
- C sin renorm post-step mientras apply usa $\hat{C}$. Reset de muertos sin CODEBOOK_RESET.
- Evaluar muerte por norma post-renorm (siempre ≈ 1).
- Un solo build.artifact_hash que no pinnea apply_ref.
- Vender el STEP como sello de θ entre SEALS, o T1 1/N como defensa adversarial.
- Atribuir un binding a attn o a mlp en fase 1.
- Declarar mag como f16. Wire sin verdict/reason.
- Baseline secuencial cuando el bloque es paralelo.
- Hashear θ completo por paso «para más seguridad».
- Jaccard de features sin alinear el codebook.
- Serializar el export con pickle. Mezclar este runtime con el foro «internet muerto».
- Prometer que las elipses del MLP se vuelven palabras. Vender Hyphae como el modelo.

Registro de correcciones

Los errores de v1 y de cada borrador quedan aquí, no se borran. Una spec que esconde su historia no puede pedir que un modelo deje la suya.

v1 → v2 (E1-E13)

#	Qué estaba mal	Cierre
E1	$H(h)+\text{binding} = H(h')$ — hashes no homomórficos	Replay T1 de h_0 + apply_ref
E2	Sin contrato de gradiente; ABORT mataba capas	STE + mask committed + aux
E3	ACK síncrono por capa en el hot path	Espejo local + barrera por paso
E4	next dentro del binding	prev opcional; head lo emite COMMIT / T2
E5	64 slots como cap de features	slot $\neq$ feature; N_f en {4k...32k}
E6	Ley 1 vacua («h empieza en x»)	Zero-init del camino de escritura
E7	Pesos invisibles	SEAL de $\theta + H(C)$ por paso
E8	ABORT de capa entera	Veredicto por candidato
E9	API Hyphae parcialmente imaginada	tx y 5 verbos existen; allocate es CAS de app
E10	Codebook sin versionar	C en la cadena del STEP
E11	Colapso de codebook omitido	Entropía + $\Sigma \text{mag} $ + reset
E12	«0,1%» y sobreclamas	Precedentes citados, no novedad de pieza
E13	Overflow como umbral falsable	Gap de PPL, T1/T2, Jaccard alineado

Addendum v2.1 (A1-A6, C1-C7)

#	Qué cerró
A1 / pack	Interfaz, k, métrica, slots vacíos
A2 / ley 0	Bloque funcional
A3 / apply	Referencia software, T1 same-binary
A4 / cadena	policy_id + C en el STEP
A5 / WAL	Cotas en el manifiesto
A6 / T1	Test de consistencia, no oráculo
C1	apply usa $\hat{C}$: $ \text{mag} $ es $\ \text{delta}\ $
C2	Baseline parallel-block (misma topología)
C3	SEAL de θ por cadencia, no hash por paso
C4	prev opcional; T2 es la cadena
C5	El binario tiene que ser un hecho
C6	T1 no es auditoría adversarial
C7	Bytes-equal; reducción fijada; abort_rules; anchor en D2H

Addendum v2.2 (C8-C15, D1-D5)

#	Qué cerró
C8	$\text{mag}==0$ ya no aborta. COMMIT + STE + elisión de WAL. Deroga C7c en esa regla.
C9	renorm post-step + min_norm + CODEBOOK_RESET
C10	Cuatro digestos en RUN_START
C11	Hueco entre SEALs = amenaza C6

#	Qué cerró
C12	El hecho nombra la suma, no attn/mlp
C13	Wire packed de 16 B
C14	#veredictos = k por token por capa
C15	alias_check como perfil
D1	Cero = no-op por regla (signo de IEEE). Elisión pre-T2.
D2	Muerte por norma se mide pre-renorm
D3	mag = bits bf16, no f16
D4	verdict y reason en el wire
D5	Dinámica del step 0 declarada; slot tautológico declarado

Cierre

Se quiere un modelo que siga entrenando a escala en B300 y MI355X, y que no pueda cambiar su estado interno — residual ni codebook, y pesos cuando se sellan — sin dejar un objeto citable, hasheable y abortable. CUDA y ROCm siguen siendo el músculo. Hyphae sigue siendo quien firma. La firma tiene una matemática que un proceso CPU puede ejecutar en frío.

Lo que falta no es otra lámina. Es apply_ref con $\hat{C}$, renorm y el no-op del cero; un bloque que no toque h; BindingMin con verdict/reason; el manifiesto con las cotas y los cuatro hashes escritos antes del step 0; y el primer run que pueda perder.

Un sistema no es confiable porque dice lo correcto. Es confiable cuando sus límites, su historia y su evidencia pueden examinarse. Aplicado a esta spec: E1–E13, A1–A6, C1–C15 y D1–D5 quedan en el registro. Ninguno en una nota al pie.

Campo	Valor
Título	El medio que falta · Hyphae en el residual — spec v2.2
Tipo	Spec de trabajo / no es un paper
Fecha	29 de agosto de 2026
Hyphae	v2.2.0 (release-v2.2.0-crates, 5bd8afb)
Precedentes	VQ-VAE · MoE / aux loss · ReZero/Fixup · SAE / Elhage · parallel block GPT-J/PaLM
Siguiente acto	apply_ref + bloque funcional + BindingMin + RUN_START